



LECTURE: 1

Search Strategies



Introduction to Search Strategies:

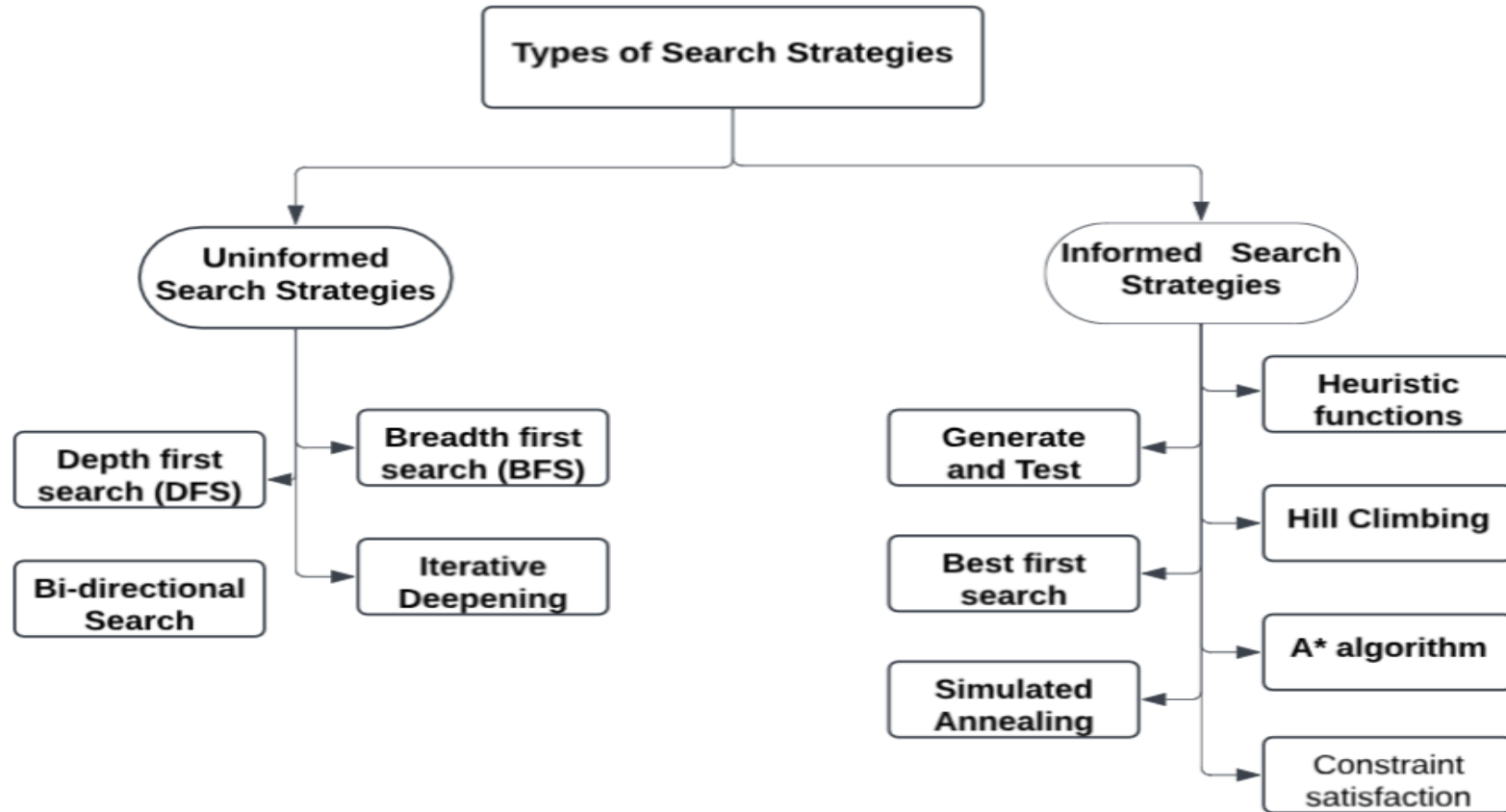
- Explore problem spaces effectively
- Aim to find solutions and goals
- Select the best path or actions
- Guide navigation through possible states
- Crucial for problem-solving, optimization, decision-making



Importance of Search Strategies:

- Solving problems: Sudoku, Google Maps
- Search programming: Code debugging, Search engines
- Goal-based agents: Siri, Autonomous vehicles
- Support production systems: Workflow Automation
- Neural network systems: Face ID, Alexa

Types of Search Strategies:





Evaluating Search Strategies:

- Completeness
- Time complexity
- Space complexity
- Optimality

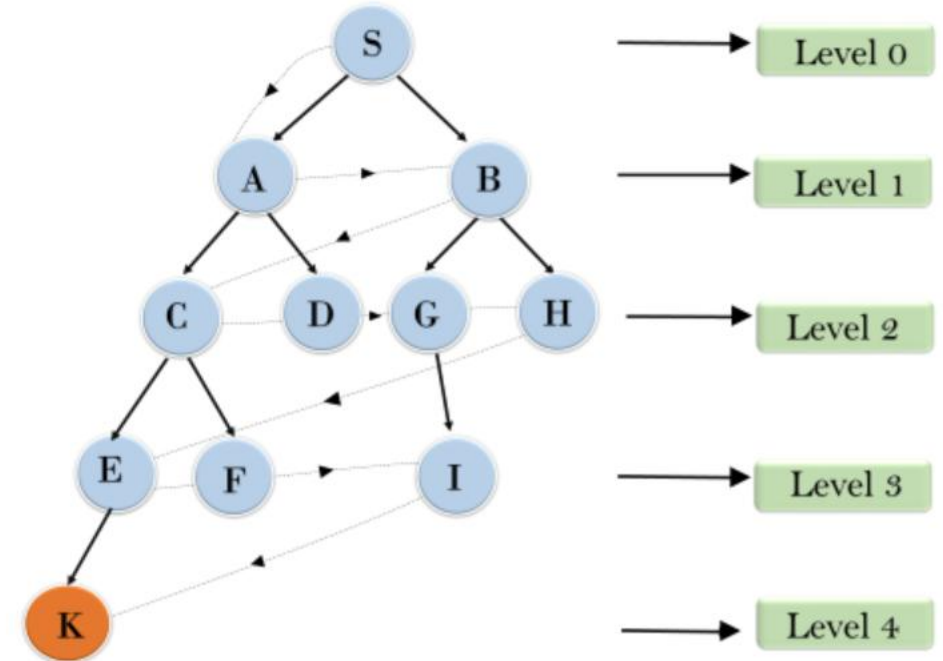
Uninformed Search Strategies:

- No additional problem-specific knowledge used
- Explore search space blindly
- Focuses on structure, not heuristics
- Guarantees a solution if it exists
- Less efficient for large problems



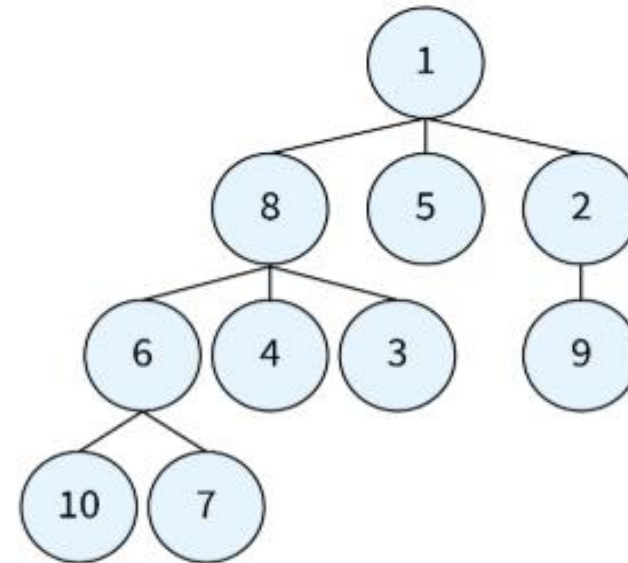
Breadth-First Search (BFS)

- Explores level by level.
- Expands nodes by increasing depth
- Find shortest path in unweighted graph
- Implemented using FIFO queue data structure.
- Time complexity: $O(V+E)$
- Space complexity: $O(V)$



Algorithm of Breadth-First Search (BFS)

- Create an empty queue, enqueue root.
- While queue isn't empty, continue process.
- Dequeue the front node from queue.
- Check if dequeued node is goal.
- If not, enqueue unvisited neighbors.
- Repeat until goal found or queue empty.



BFS : 1 8 5 2 6 4 3 9 10 7



Pseudocode for BFS:

Input: s as the source node

```
BFS (G, s)
let Q be queue.
Q.enqueue( s )
```

```
mark s as visited
while ( Q is not empty)
v = Q.dequeue( )
```

```
for all neighbors w of v in Graph G
if w is not visited
Q.enqueue( w )
mark w as visited
```

Advantages of BFS:

- Finds the shortest path fast
- Explores all possibilities
- Simple and easy to implement
- Guarantees complete solution
- Good for shallow searches
- Suitable for unweighted graphs



Disadvantages of BFS:

- High memory consumption
- Slow for large graphs
- Inefficient for weighted graphs
- Requires additional storage space
- Fails with infinite paths
- Expensive computationally at scale

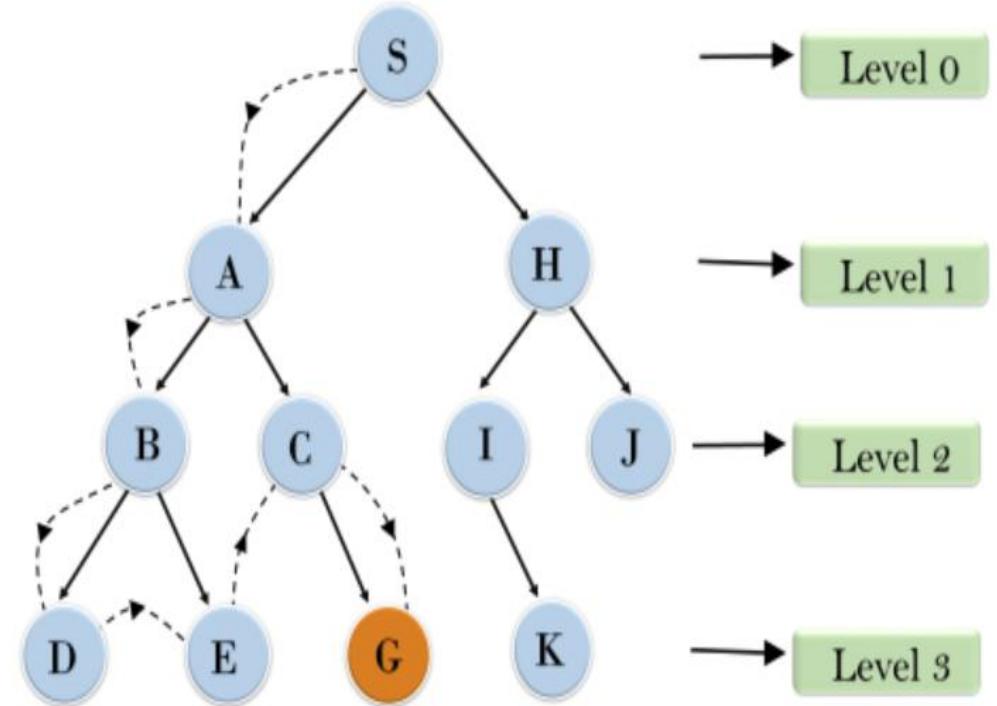


Task: Find path from City A City G.



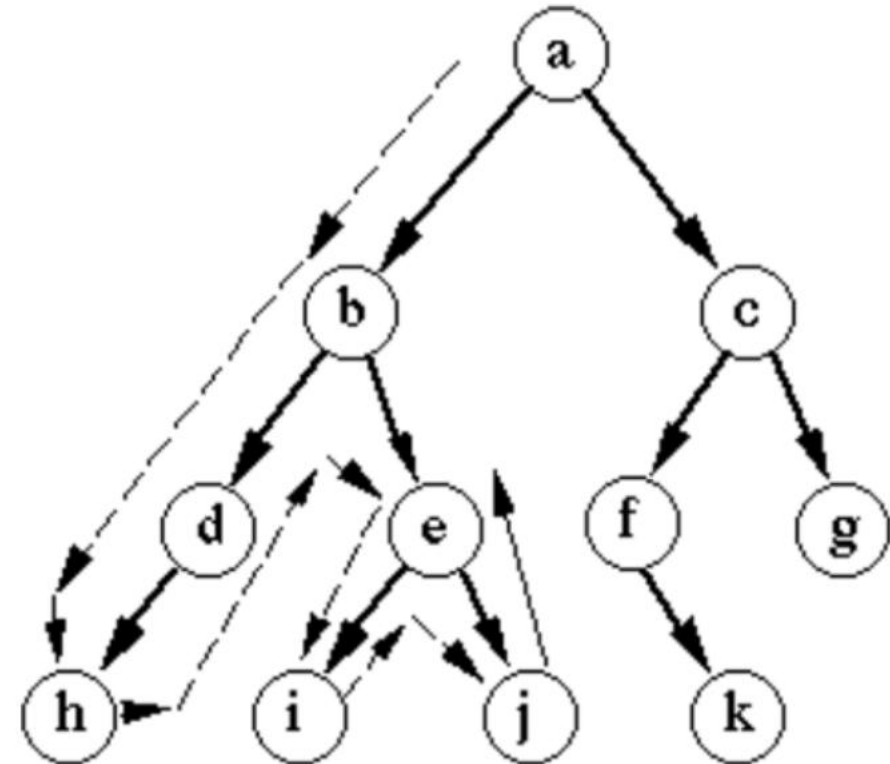
Depth-First Search (DFS)

- Explores deeply before backtracking.
- Uses a stack for tracking.
- Time complexity: $O(V+E)$
- Space complexity: $O(V)$
- Not guaranteed to find optimal path.
- May get stuck in infinite loops.



Algorithm of Depth-First Search (DFS)

- Create an empty stack, push root.
- While stack isn't empty, continue process.
- Pop the top node from stack.
- Check if popped node is goal.
- If not, push unvisited neighbors.
- Repeat until goal found or stack empty.



Pseudocode for DFS:

```
DFS (root node) {  
    create stack  
    create list of visited nodes  
    mark root node as visited  
    push root node into stack  
    while (stack is not empty) {  
        x = stack.top()  
        stack.pop()  
        for (all immediate neighbors of x) {  
            if (not visited) {  
                push into stack  
                mark as visited  
            }  
        }  
    }  
}
```



Advantages of DFS:

- Low memory usage
- Works well with deep graphs
- Simple and easy to implement
- Efficient for cyclic graphs
- Suitable for pathfinding in mazes
- Less storage space required

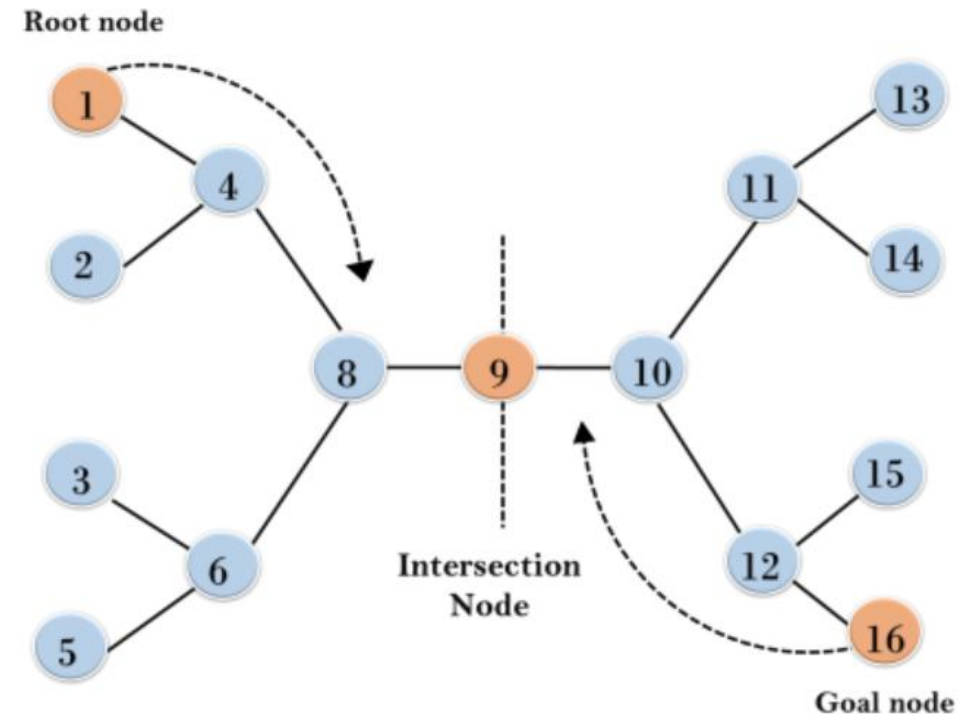


Disadvantages of DFS:

- May not find shortest path
- Not guaranteed to find all solutions
- High time consumption in deep paths
- Fails in graphs with loops
- Can get stuck in infinite paths

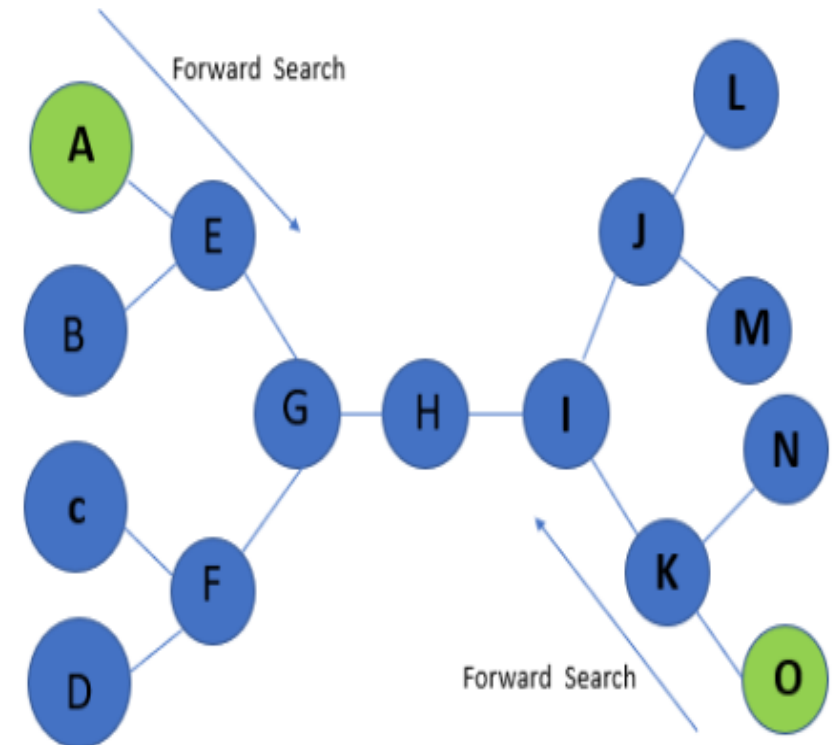
Bi-Directional Search

- Searches from start and goal simultaneously
- Meets in the middle for efficiency
- Faster for large search spaces
- Time complexity: $O(b^{d/2})$
- Space complexity: $O(b^{d/2})$
- Eg: Google Maps, Social networks, Gaming



Algorithm of Bi-Directional Search

- Initialize two queues for searches.
- Start forward and backward searches simultaneously.
- Expand nodes from both search fronts.
- Check if nodes meet in middle.
- If paths connect, return the solution.
- Repeat until solution found or queues empty.





Advantages of Bi-Directional Search

- Reduces search space significantly.
- Faster than unidirectional search.
- Efficient for large state spaces.
- Can find optimal solutions.
- Improves search time in complex networks.
- requires less memory than traditional algorithms



Disadvantages of Bi-Directional Search

- Requires clearly defined start and goal.
- More complex to implement.
- Needs synchronization between two search fronts.
- Memory usage can be high in some cases.
- Less effective in asymmetric graphs.
- Struggles with non-uniform transition costs.

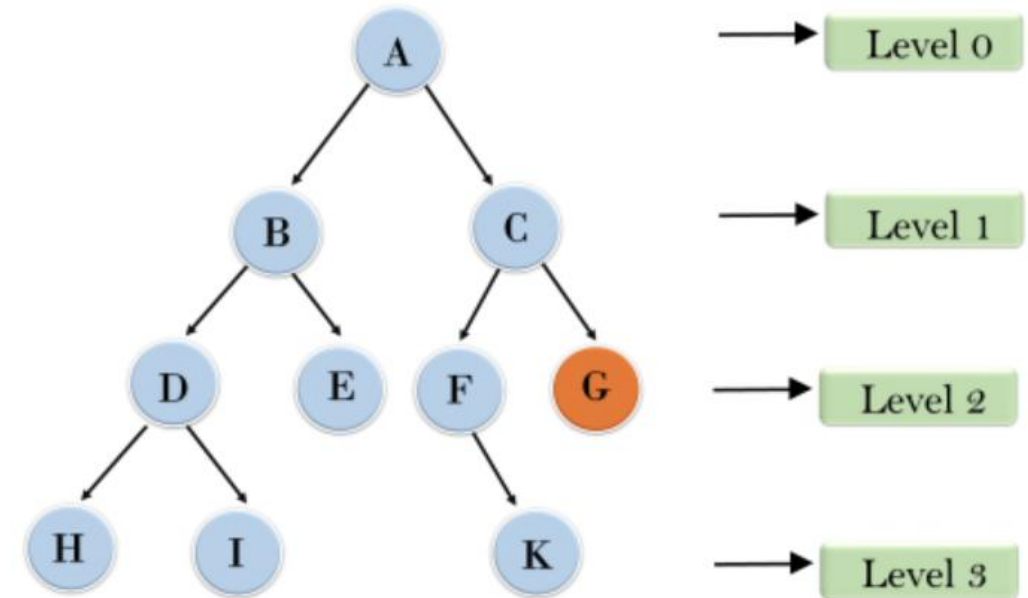


Quiz Time

- _____ is used for implementing DFS?
 - a) Queue
 - b) Stack
 - c) Heap
 - d) All of the above

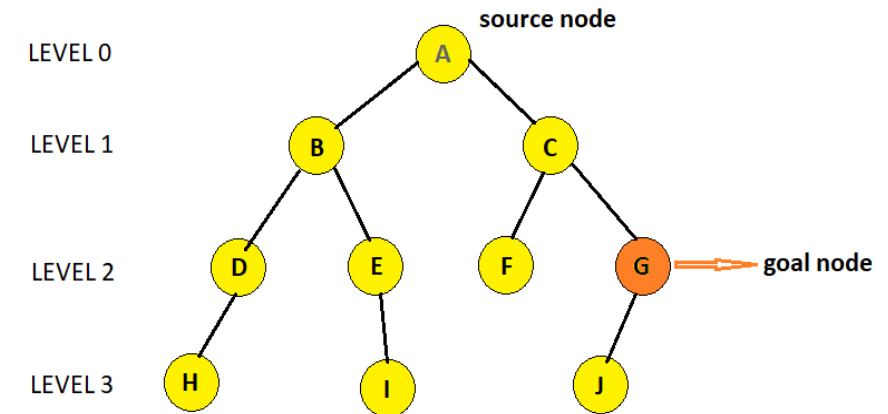
Iterative Deepening Search (IDS)

- Combines DFS and BFS features.
- Repeated DFS with increasing depth limits.
- Suitable for memory-constrained problems
- Time complexity: $O(b^d)$
- Space complexity: $O(bd)$
- Eg: Puzzle Solving, Game AI, Robotics



Algorithm of Iterative Deepening Search (IDS)

- Set initial depth limit to zero.
- Perform Depth-Limited Search (DLS).
- Increment depth limit after each iteration.
- Continue DLS within updated depth limit.
- Check if goal node is found.
- Repeat until goal found or limit reached.



IDDFS with max depth-limit = 3

Note that iteration terminates at depth-limit=2

Iteration 0: A

Iteration 1: A->B->C

Iteration 2: A->B->D->E->C->F->G



Pseudo-code for IDDFS:

IterativeDeepeningSearch(root, goal)

 for depth = 0 to ∞ do

 result = DepthLimitedSearch(root, goal, depth)

 if result == found then

 return result



Advantages of IDS:

- Memory efficient like Depth-First Search.
- Guarantees finding the optimal solution.
- Avoids getting stuck in infinite loops.
- Combines benefits of DFS and BFS.
- Suitable for deep, unknown search spaces.
- Expands shallow nodes multiple times effectively.



Disadvantages of IDS:

- Repeated work in re-expanding nodes.
- Slower than BFS for shallow solutions.
- Higher computational cost due to repetition.
- Not ideal for high branching factors.
- Inefficient for problems with uniform costs.
- Requires goal state to be predefined.



Advantages of Uninformed Search Strategies:

- Simple and easy to implement.
- Requires no prior domain knowledge.
- Guarantees finding a solution, if exists.
- Suitable for small search spaces.
- Systematic exploration of all possibilities.
- Can be applied to diverse problems.

Disadvantages of Uninformed Search Strategies:

- Inefficient for large search spaces.
- High time and memory requirements.
- Lacks optimization for best solutions.
- Explores irrelevant or redundant paths.
- Cannot handle infinite search spaces.
- Fails in complex or dynamic problems.



Quiz Time

- **Which search strategy may loop infinitely?**
 - a) Breadth-First Search
 - b) Depth-First Search
 - c) Uniform Cost Search
 - d) Iterative Deepening Search



Quiz Time

- **Which is NOT an uninformed search?**
 - a) Breadth-First Search
 - b) Depth-First Search
 - c) Uniform Cost Search
 - d) A* Search